TECNOLOGIAS E BANCO DE DADOS

PROJETO APLICATIVO DE TÊNIS

Gabriel Castro, Eduardo Estevam

Sumário

1. Opções de Frontend 2
1.1 Flutter 2
1.2 React Native 3
1.3 Kotlin 4
2. Opções de Backend 6
2.1 Python 6
2.1 Javascript 6
2.2 Java 7
3. Opções de banco de dados 7
3.1 PostgreSQL 7
3.2 MySQL 8
4. Justificativa da stack escolhida 9
4.1 Flutter no Frontend 9
4.2 Python no Backend e Integração com Supabase 10
4.3 PostgreSQL como Banco de Dados 10
5. Bibliografia 12

1 - Opções de Frontend

Comparação entre 3 principais concorrentes e escolha final.

1.1 Flutter

O que é: O Flutter é uma estrutura de código aberto desenvolvida e sustentada pelo Google. Desenvolvedores de front-end e full-stack usam o Flutter para criar a interface de usuário (UI) de uma aplicação para várias plataformas com uma única base de código.

Características: O Flutter usa a linguagem de programação de código aberto Dart, que também foi desenvolvida pelo Google. A linguagem Dart é otimizada para o desenvolvimento de interfaces de usuário e muitos dos pontos fortes dela são usados no Flutter.

Por exemplo, um recurso da linguagem Dart usado no Flutter é a segurança nula de som. A segurança nula de som da linguagem Dart facilita a detecção de bugs comuns, chamados de null errors (erros nulos). Esse recurso reduz o tempo que os desenvolvedores gastam na manutenção do código e dá a eles mais tempo para se concentrar no desenvolvimento de suas aplicações.
Além disso, o Flutter possui diversos Widgets prontos que tornam o desenvolvimento muito mais rápido e versátil.

E por fim, um único código pode compilar para diversas plataformas, como Windows, Android, Mac, Linux e IOS.

Vantagens:

- A performance é próxima à nativa. O Flutter usa a linguagem de programação Dart e realiza o compilamento em código de máquina. Os dispositivos de host entendem esse código, o que garante uma performance rápida e eficaz.
- A renderização é rápida, consistente e personalizável. Em vez de depender de ferramentas de renderização específicas da plataforma, o Flutter usa a biblioteca gráfica Skia de código aberto do Google para renderizar a interface do usuário. Isso fornece aos usuários visuais consistentes, independentemente da plataforma que eles usam para acessar uma aplicação.
- As ferramentas são de fácil utilização para os desenvolvedores. O Google desenvolveu o Flutter com ênfase na facilidade de uso. Com ferramentas como a recarga frequente, os desenvolvedores podem visualizar como serão as alterações de código sem perder o estado. Outras ferramentas, como o inspetor de widgets, facilitam a visualização e a solução de problemas com layouts da interface do usuário.

Desvantagens:

- **Tamanho do arquivo APK:** O tamanho final do arquivo APK pode ser maior do que em versões nativas devido à presença da engine Flutter no

aplicativo, afinal esse é um dos pontos que pode afetar o espaço de armazenamento nos dispositivos dos usuários finais.

- **Dependência de packages:** O Flutter depende fortemente de pacotes construídos pela comunidade ou por terceiros logo isso pode resultar em atrasos na implementação de novas funcionalidades dos sistemas operacionais e incompatibilidades entre pacotes. Embora o Flutter ofereça várias vantagens significativas para os desenvolvedores, por outro lado é importante considerar suas desvantagens antes de decidir usá-lo em um projeto. Ainda assim, suas características e funcionalidades únicas podem oferecer uma vantagem competitiva significativa no desenvolvimento de aplicativos móveis.

1.2 React Native

O que é: O React Native é um framework de código aberto para desenvolvimento mobile, criado pelo Facebook, que permite criar aplicações multiplataforma nativas para iOS e Android usando JavaScript. Empresas como Uber, Microsoft e Facebook utilizam o framework React Native em seus aplicativos, mostrando sua robustez.

Características: O React Native foi lançado em 2015 pelo Facebook como um projeto de código aberto. Em apenas alguns anos, tornou-se uma das principais soluções adotadas no desenvolvimento móvel, e hoje é utilizada por aplicativos mundialmente famosos como o Discord, Tesla, Instagram, Facebook e entre outros. Discutiremos esses e outros exemplos de aplicativos baseados em React Native mais adiante neste post.

Dentre as várias razões por trás do sucesso do React Native, a primeira delas é que as empresas podem criar o código apenas uma vez e usá-lo para alimentar seus aplicativos iOS e Android - e isso se traduz em uma enorme economia de tempo e recursos.

Vantagens:

- Experiência do usuário: a experiência da pessoa que utiliza um App feito em React Native é tão boa quanto a programação nativa, visto que os códigos, sejam em JavaScript ou TypeScript, serão convertidos em linguagem nativa, o deixando bem performático;
- Tempo: é bem mais rápido validar uma ideia no mercado, quando se usa o React Native para obter feedbacks e adaptar as mudanças, sem a necessidade de um grande investimento;
- Comunidade: a comunidade React Native é enorme. Com isso, a pessoa desenvolvedora consegue resolver problemas de uma forma bem rápida;
- Custo de manutenção: você lida apenas com uma base de código.

Desvantagens:

- Desempenho menor que o nativo: React Native não é capaz de usar todas as vantagens e o potencial de uma plataforma específica. Assim, um aplicativo nativo pode realmente maximizar as funcionalidades e entregar a melhor experiência ao usuário no resultado final;
- Módulos personalizados: muitos módulos personalizados já estão disponíveis, mas pode haver a necessidade de alguns componentes específicos que você precisará construir do zero;
- Problemas de atualização: é difícil manter o aplicativo atualizado com a versão mais recente do React Native. Atualizar versões do React Native é, na maioria dos casos, um processo complicado;
- Recursos novos: os recursos novos que chegam aos nativos podem demorar a chegar para o React Native.

1.3 Kotlin

O que é: Kotlin é uma linguagem de programação orientada a objetos compatível com a plataforma Java e que pode ser executada em máquinas virtuais Java (JVM), além de outras plataformas, como o Android. Além disso, Kotlin tem uma sintaxe semelhante ao Java e muitas das suas características e conceitos de programação são parecidas.

É uma linguagem moderna que pode ser utilizada para desenvolver aplicativos Android, web e desktop. Conhecida também como uma linguagem de programação prática, ela foi projetada para ser segura e concisa, assim permite que pessoas desenvolvedoras escrevam menos código para realizar tarefas simples.

Caractéristicas: Como é uma das linguagens derivadas do Java, o Kotlin tem como sua principal característica rodar no ecossistema Java: seu código compila para bytecode, que é interpretado pela Java Virtual Machine, assim como acontece em sua linguagem-mãe. Essa propriedade possibilita que o Kotlin aproveite código escrito em Java em suas aplicações também, ou seja, existe interoperabilidade com as duas linguagens.

Sendo assim, o Kotlin pode utilizar bibliotecas escritas em sua linguagem modelo e reaproveitá-las sem a necessidade de escrever o código do zero, o que contribui, inclusive, para que a linguagem seja tão robusta quanto o Java.

Vantagens:

- Menos é mais: uma das grandes vantagens dessa linguagem é a criação de código menos verbosos. Ela oferece um código mais limpo e curto em comparação com a linguagem Java (a qual Kotlin é interoperável). Com isso, o código fica mais legível e também agiliza o processo de programação.

- Quem nunca teve erro de ponto e vírgula: Kotlin dispensa o uso de ponto e vírgula, mas também não reclama quando os vícios de linguagem Java falam mais alto.
- Dessa forma, há uma menos possibilidade de bugs, e diminui as chances de erros em tempo de execução e compilação.
- Interoperável com Java: Kotlin permite a utilização das bibliotecas e frameworks Java. Assim, todo conhecimento acumulado com o desenvolvimento em Java não é desperdiçado.
- Null safety: sistema de tipos do Kotlin tem como objetivo eliminar o famoso NullPointerException. Mas há algumas possibilidades de alertas de NPEs, então é bom ficar alerta.

Desvantagens:

- Flutuação na velocidade de compilação: nesse quesito Java apresenta uma vantagem em relação ao Kotlin.
- Falta de fonte de conhecimento: por ser uma linguagem relativamente nova, não há tanto recursos de aprendizagem no mercado.
- Compartilhamento: a comunidade de suporte ainda é pequena, em comparação com Java, há poucos tutoriais e ajudas disponíveis nessa linguagem.
- As desvantagens podem ser desafiadoras, mas desistir não pode ser considerada uma opção.

2 - Opções de Backend

Comparação entre 3 principais concorrentes e escolha final.

2.1 Python

O que é: Uma linguagem reconhecida por sua simplicidade e clareza, sendo amplamente adotada em cursos e treinamentos para ajudar novos programadores a compreenderem lógica e estruturas de código. Além disso, destaca-se fortemente na área de dados, em inteligência artificial e no desenvolvimento back-end.

Características: A linguagem opera no ecossistema de dados por meio de bibliotecas como Pandas e NumPy para manipulação de grandes volumes de informação, e utiliza frameworks como TensorFlow, Keras e PyTorch no desenvolvimento de modelos de inteligência artificial. No desenvolvimento web, emprega Django e Flask para estruturar serviços e APIs, além de marcar presença na criação de jogos e protótipos multimídia via Pygame, em modelos quantitativos para o mercado financeiro e fintechs, e em projetos de georreferenciamento.

Vantagens: Proporciona grande agilidade na criação de APIs e serviços web robustos por meio de seus frameworks. Facilita drasticamente a limpeza, transformação e visualização de dados em poucos comandos, bem como o aprendizado de máquinas, servindo tanto a programadores iniciantes quanto a cientistas de dados e engenheiros de IA.

Desvantagens: Possui menor velocidade de execução e desempenho computacional bruto quando comparada a linguagens compiladas (como Java ou C++), devido à sua natureza interpretada. Além disso, o Global Interpreter Lock (GIL) pode limitar a execução paralela de threads em processadores multi-core, e o consumo de memória costuma ser mais elevado.

2.2 Javascript

O que é: Uma linguagem essencial para o ecossistema web, projetada para fazer as páginas ganharem vida e interatividade. Com a chegada do Node.js, ela ultrapassou o navegador e expandiu-se para o servidor, viabilizando o desenvolvimento full-stack utilizando uma única linguagem.

Características: No lado do cliente, lida com cliques, formulários, animações, validações e atualizações em tempo real. Conta com bibliotecas e frameworks como React, Vue e Angular para interfaces web modernas e responsivas, além do React Native para aplicativos móveis. Permite também o desenvolvimento de aventuras gráficas, visualizações em 3D e jogos interativos no navegador através de ferramentas como Three.js e Phaser.js.

Vantagens: Permite construir aplicações full-stack completas — unindo front-end, mobile e back-end — utilizando unicamente o JavaScript. Oferece versatilidade incomparável para lidar com experiências dinâmicas de usuário, atualizações instantâneas e um vasto ecossistema de pacotes (npm).

Desvantagens: A tipagem dinâmica e fraca pode levar a erros inesperados em tempo de execução se não houver um rigoroso controle ou o uso de ferramentas adicionais como o TypeScript. O modelo assíncrono baseado em callbacks, embora eficiente para I/O, pode gerar o problema conhecido como "callback hell" (código aninhado complexo) caso a assincronicidade não seja bem gerenciada.

2.3 Java

O que é: Uma tecnologia consolidada e de grande popularidade, tradicionalmente valorizada em ambientes corporativos e sistemas empresariais de longa duração por equilibrar de forma sólida o desempenho, a confiabilidade e a escalabilidade.

Características: Sua arquitetura baseia-se na execução por meio da Máquina Virtual Java (JVM), que realiza o gerenciamento eficiente de memória e aplica otimizações avançadas em tempo de execução, como a compilação JIT (Just-In-Time). Essa base garante total portabilidade entre diferentes sistemas operacionais e ambientes de infraestrutura.

Vantagens: Oferece desempenho superior em tempo de execução e eficiência em comparação a linguagens interpretadas, especialmente em sistemas complexos e de grande escala. Além disso, garante total independência de plataforma, rodando sem atrito em Windows, Linux, macOS e em diversos ambientes de computação em nuvem.

Desvantagens: Apresenta uma curva de aprendizado mais acentuada e exige uma quantidade maior de código (boilerplate) para realizar tarefas simples em comparação ao Python ou JavaScript. O consumo de memória inicial costuma ser alto devido à exigência da Máquina Virtual Java (JVM) para rodar as aplicações.

3 - Opções de Banco de Dados

Pesquisar, comparar e escolher o Banco de Dados (mínimo 2 opções)

3.1 PostgreSQL

O que é: O PostgreSQL é um sistema de banco de dados objeto/relacional altamente avançado, projetado para lidar com cargas de trabalho corporativas complexas, integridade estrita de dados e extensibilidade profunda através de múltiplos recursos de engenharia de software. No geral, o PostgreSQL é um SGBD de arquitetura objeto/relacional que estende o modelo relacional padrão com suporte a herança de tabelas, tipos complexos (como matrizes, JSONB e

XML) e execução de lógica procedural avançada diretamente no banco de dados.

Características: Conformidade rigorosa com ACID e controle de concorrência multiversão MVCC nativos e imutáveis em absolutamente todas as configurações e mecanismos do sistema, suporte sofisticado a índices parciais, de expressão, de hash e árvores avançadas, visões materializadas para pré-cálculo e cache de consultas analíticas pesadas, procedimentos armazenados escritos em múltiplas linguagens (incluindo Python, PL/pgSQL e Perl) e gatilhos avançados do tipo INSTEAD OF para interceptação e manipulação complexa de visualizações.

Vantagens: Extrema robustez para arquiteturas modernas, integração conceitual fluida com linguagens orientadas a objetos, suporte nativo a tipos flexíveis que facilitam a serialização de dados consumidos por APIs em Python (como FastAPI ou Django REST Framework) para frontends multiplataforma em Flutter (web e mobile), além de visões materializadas que aceleram drasticamente painéis analíticos na versão web.

Desvantagens: Curva de aprendizado inicial mais acentuada para desenvolvedores e administradores de sistemas devido à vastidão de recursos avançados, e maior consumo de recursos computacionais de hardware para gerenciar estruturas complexas de cache e processamento de consultas paralelas pesadas.

3.2 MySQL

O que é: O MySQL é um sistema de gerenciamento de banco de dados relacional clássico baseado em tabelas tabulares, amplamente consolidado no ecossistema de servidores web e arquiteturas tradicionais cliente-servidor, otimizado para operações rápidas de leitura e escrita em estruturas plana-relacionais. No geral, o MySQL é um SGBD estritamente relacional estruturado em linhas e colunas, amplamente utilizado em pilhas tecnológicas tradicionais para alimentar páginas web dinâmicas, sistemas integrados e aplicações orientadas a transações simples.

Características: Dependência estrita do mecanismo de armazenamento InnoDB para garantir transações ACID e controle de concorrência multiversão MVCC por linha, suporte a índices B-tree e R-tree para buscas hierárquicas e espaciais, exibições padrão, procedimentos armazenados executados em sintaxe SQL pura e gatilhos (triggers) restritos exclusivamente aos eventos AFTER e BEFORE nas operações de inserção, atualização e exclusão.

Vantagens: Alta popularidade global, vasta documentação técnica, excelente ecossistema de hospedagem de baixo custo e simplicidade de integração via drivers e ORMs em Python (como PyMySQL ou SQLAlchemy) para alimentar APIs que atendem aplicações web e mobile em Flutter através de um backend intermediário.

Desvantagens: Ausência de conformidade ACID nativa universal (visto que motores legados como o MyISAM carecem totalmente de suporte a transações e MVCC), suporte limitado a tipos de dados avançados, restrições severas na criação de gatilhos complexos sem a opção INSTEAD OF, e incapacidade de processar nativamente paradigmas de orientação a objetos ou herança de tabelas.

4 – Justificativa da stack escolhida.

Escrever documento de justificativa da stack (1-2 páginas com os "porquês")

4.1 Flutter no Frontend: A adoção do Flutter como tecnologia de frontend se justifica por sua alta produtividade, excelente desempenho e forte consolidação no mercado. Atualmente, ele é o framework multiplataforma mais utilizado no mundo, superando alternativas como o React Native, segundo dados da Stack Overflow Developer Survey e da Statista.

Utilizando a linguagem Dart — reconhecida por sua baixa curva de aprendizado, forte tipagem e tratamento nativo de assincronismo —, o ecossistema do Flutter oferece uma vasta biblioteca de widgets nativamente responsivos e altamente customizáveis. Isso acelera a construção de interfaces complexas e garante total fidelidade visual entre diferentes dispositivos e tamanhos de tela.

Principais Pilares Técnicos e Estratégicos:

- **Performance Gráfica Nativa (AOT e Impeller):** Diferente de soluções que utilizam pontes (bridges) de comunicação intermediárias, o Flutter compila o código diretamente para linguagem de máquina (AOT - Ahead-Of-Time) e desenha cada pixel na tela utilizando sua própria engine gráfica (Impeller/Skia). Isso garante taxas de quadros fluidas (60 a 120 FPS) e tempos de resposta instantâneos, essenciais para exibição de dados dinâmicos como chaves de campeonatos, mapas e listas em tempo real.
- **Produtividade Acelerada com Stateful Hot Reload:** Permite que alterações no código e ajustes de interface sejam visualizados instantaneamente no dispositivo sem perder o estado atual da aplicação, otimizando o ciclo de testes e aumentando drasticamente a velocidade de desenvolvimento.
- **Redução de Custos e Ciclo de Vida (Single Codebase):** A capacidade de compilar para Android e iOS a partir de um único código-fonte elimina a necessidade de manter duas equipes distintas de desenvolvimento. Isso reduz o tempo de lançamento (time-to-market), simplifica o processo de garantia de qualidade (QA) e reduz em até 50% os custos de manutenção contínua e correção de bugs.

- **Integração Eficiente e Segurança:** A linguagem Dart possui suporte nativo para manipulação de coleções, dados JSON e requisições HTTP assíncronas, facilitando o consumo de APIs REST/GraphQL no backend. Além disso, a segurança do recurso Null Safety impede erros em tempo de execução (runtime crashes) causados por valores nulos, aumentando a estabilidade geral da aplicação.

4.2 Python no Backend e Integração com Supabase: A adoção do Python para a camada de backend fundamenta-se em sua reconhecida produtividade, sintaxe expressiva e baixa curva de aprendizado, fatores que reduzem consideravelmente o tempo de desenvolvimento (time-to-market) e facilitam a manutenção contínua do código. Trata-se de uma linguagem amplamente consolidada na indústria, com suporte massivo da comunidade e um ecossistema maduro para construção de arquiteturas escaláveis.

Principais Justificativas Técnicas:

- **Construção de APIs Performáticas:** A linguagem conta com frameworks modernos como FastAPI e Flask, que viabilizam a criação de APIs RESTful de forma modular, segura e bem documentada (com suporte nativo à especificação OpenAPI/Swagger). O suporte a operações assíncronas (async/await) no Python garante alta vazão no processamento de requisições concorrentes, essencial para gerenciar simultaneamente alunos, turmas e eventos do sistema.
- **Integração Eficiente com Supabase (BaaS):** A escolha do Supabase como Backend-as-a-Service traz uma camada de banco de dados relacional PostgreSQL de alta performance. O Python possui SDKs nativas e altamente otimizadas para o Supabase, permitindo realizar operações de CRUD (Criação, Leitura, Atualização e Deleção), consultas complexas e escuta de eventos em tempo real (Websockets) com extrema simplicidade e poucas linhas de código.
- **Autenticação e Segurança Embutidas:** Ao conectar o Python à estrutura do Supabase, o sistema herda nativamente mecanismos avançados de autenticação (JWT) e políticas de segurança em nível de linha (Row Level Security - RLS), garantindo que os dados de campeonatos, notas e perfis fiquem isolados e protegidos por padrão.
- **Tipagem e Legibilidade:** Com o suporte ao Type Hinting (tipagem estática opcional) nas versões modernas do Python, é possível reduzir drasticamente falhas em tempo de execução (bugs em runtime), facilitando a validação de dados de entrada e a integração perfeita com as telas do aplicativo em Flutter.

4.3 PostgreSQL como Banco de Dados: A opção pelo PostgreSQL em detrimento do MySQL fundamenta-se em sua arquitetura orientada a objetos-relacional, robustez técnica e conformidade rigorosa com os padrões ACID. Para uma aplicação multiplataforma (Web e Mobile Flutter) integrada a um backend Python, o PostgreSQL oferece recursos avançados que superam as

capacidades do MySQL em termos de escalabilidade, extensibilidade e segurança.

Principais Justificativas Técnicas:

- Suporte Nativo a JSONB e Dados Semiestruturados: Diferente do MySQL, o PostgreSQL oferece o tipo de dado JSONB, que armazena dados em formato binário decomposto. Isso permite indexação avançada (utilizando índices GIN) e consultas de altíssima performance em documentos JSON, viabilizando flexibilidade para recursos do aplicativo que exigem esquemas dinâmicos sem abrir mão do rigor relacional.
- Segurança em Nível de Linha (Row Level Security - RLS): O PostgreSQL possui suporte nativo granular a RLS, permitindo definir políticas de acesso diretamente na camada de banco de dados. Esse recurso é crucial para isolar com precisão os dados dos usuários entre as sessões mobile e web, reduzindo a complexidade de validação de segurança no backend Python.
- Recursos Avançados de Concorrência (MVCC): A implementação do controle de concorrência multiversão (MVCC) no PostgreSQL lida de forma superior com altas taxas de leituras e escritas simultâneas. Isso previne bloqueios de tabela e garante consistência e fluidez de resposta tanto na interface web quanto no aplicativo Flutter durante picos de acessos concorrentes.
- Integração Perfeita com o Ecossistema Python e BaaS: O PostgreSQL possui o ecossistema de drivers e ORMs mais maduro do mercado para Python (como SQLAlchemy, psycopg3 e Tortoise-ORM). Além disso, por ser o motor fundamental de soluções modernas de Backend-as-a-Service (como o Supabase), sua escolha viabiliza o uso direto de extensões poderosas (como pgvector para inteligência artificial ou PostGIS para geolocalização) sem necessidade de migração de banco no futuro.

5. Bibliografia:

O que é Flutter? – Explicação sobre a aplicação Flutter – AWS

Flutter: Vantagens e Desvantagens na Criação de Aplicativos - ELO Júnior

React Native: o que é, como funciona o framework e tudo sobre desenvolvimento mobile | Alura Cursos Online

Linguagem Kotlin: o que é, para que serve e um Guia para aprender | Alura Cursos Online

Vantagens e Desvantagens do Kotlin | Débora Silva | Kotlin | DIO

Python vs. Java: Qual linguagem você deve escolher? (2026) - LogicMonitor

Python ou JavaScript: qual linguagem aprender primeiro e por quê

PostgreSQL vs. MySQL | Diferença entre sistemas de gerenciamento de bancos de dados relacionais (RDBMS) | AWS